

# MLA0303\_RL\_PROGRAM\_3 — Program with Output

## Python Source Code

```
"""
Experiment 3: MDP design for an Autonomous Warehouse Robot.
This experiment focuses on *designing* the MDP: states, actions,
stochastic transition probabilities and the reward function.
"""

GRID_ROWS, GRID_COLS = 4, 4
SHELF = (0, 3)      # pickup location
STATION = (3, 0)     # drop-off / packing station

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT", "PICK", "DROP"]
MOVE_DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
PERP = {"UP": ["LEFT", "RIGHT"], "DOWN": ["LEFT", "RIGHT"],
        "LEFT": ["UP", "DOWN"], "RIGHT": ["UP", "DOWN"]}

STEP_COST = -1
PICK_REWARD = 5
DROP_REWARD = 20
INVALID_TASK_PENALTY = -3

def in_bounds(pos):
    r, c = pos
    return 0 <= r < GRID_ROWS and 0 <= c < GRID_COLS

def all_states():
    states = []
    for r in range(GRID_ROWS):
        for c in range(GRID_COLS):
            for carrying in (False, True):
                states.append((r, c, carrying))
    return states

def transitions(state, action):
    """Return list of (probability, next_state, reward)."""
    r, c, carrying = state
    pos = (r, c)

    if action in MOVE_DELTA:
        outcomes = []
        dirs = [(action, 0.8)] + [(p, 0.1) for p in PERP[action]]
        for a, prob in dirs:
            dr, dc = MOVE_DELTA[a]
            new_pos = (r + dr, c + dc)
            if not in_bounds(new_pos):
                new_pos = pos # bump into wall, stay in place
            outcomes.append((prob, (new_pos[0], new_pos[1], carrying), STEP_COST))
        return outcomes

    if action == "PICK":
        if pos == SHELF and not carrying:
            return [(1.0, (r, c, True), PICK_REWARD)]
        return [(1.0, state, INVALID_TASK_PENALTY)]

    if action == "DROP":
        if pos == STATION and carrying:
            return [(1.0, (r, c, False), DROP_REWARD)] # terminal
        return [(1.0, state, INVALID_TASK_PENALTY)]

    raise ValueError("Unknown action")

def is_terminal(state):
    r, c, carrying = state
    return (r, c) == STATION and not carrying

if __name__ == "__main__":
    states = all_states()
    print(f"Total states          : {len(states)}")
    print(f"Actions                  : {ACTIONS}")
    print(f"Shelf (pickup) location : {SHELF}")
    print(f"Station (drop) location : {STATION}\n")

    sample_states = [(0, 0, False), (0, 3, False), (3, 0, True)]
    for s in sample_states:
```

```
print(f"State {s}:")
for a in ACTIONS:
    outcomes = transitions(s, a)
    print(f"    action={a:<6} -> {outcomes}")
print()
```

# Execution Output

```
Total states          : 32
Actions               : ['UP', 'DOWN', 'LEFT', 'RIGHT', 'PICK', 'DROP']
Shelf (pickup) location : (0, 3)
Station (drop) location : (3, 0)
```

```
State (0, 0, False):
  action=UP      -> [(0.8, (0, 0, False), -1), (0.1, (0, 0, False), -1), (0.1, (0, 1, False), -1)]
  action=DOWN    -> [(0.8, (1, 0, False), -1), (0.1, (0, 0, False), -1), (0.1, (0, 1, False), -1)]
  action=LEFT    -> [(0.8, (0, 0, False), -1), (0.1, (0, 0, False), -1), (0.1, (1, 0, False), -1)]
  action=RIGHT   -> [(0.8, (0, 1, False), -1), (0.1, (0, 0, False), -1), (0.1, (1, 0, False), -1)]
  action=PICK    -> [(1.0, (0, 0, False), -3)]
  action=DROP    -> [(1.0, (0, 0, False), -3)]
```

```
State (0, 3, False):
  action=UP      -> [(0.8, (0, 3, False), -1), (0.1, (0, 2, False), -1), (0.1, (0, 3, False), -1)]
  action=DOWN    -> [(0.8, (1, 3, False), -1), (0.1, (0, 2, False), -1), (0.1, (0, 3, False), -1)]
  action=LEFT    -> [(0.8, (0, 2, False), -1), (0.1, (0, 3, False), -1), (0.1, (1, 3, False), -1)]
  action=RIGHT   -> [(0.8, (0, 3, False), -1), (0.1, (0, 3, False), -1), (0.1, (1, 3, False), -1)]
  action=PICK    -> [(1.0, (0, 3, True), 5)]
  action=DROP    -> [(1.0, (0, 3, False), -3)]
```

```
State (3, 0, True):
  action=UP      -> [(0.8, (2, 0, True), -1), (0.1, (3, 0, True), -1), (0.1, (3, 1, True), -1)]
  action=DOWN    -> [(0.8, (3, 0, True), -1), (0.1, (3, 0, True), -1), (0.1, (3, 1, True), -1)]
  action=LEFT    -> [(0.8, (3, 0, True), -1), (0.1, (2, 0, True), -1), (0.1, (3, 0, True), -1)]
  action=RIGHT   -> [(0.8, (3, 1, True), -1), (0.1, (2, 0, True), -1), (0.1, (3, 0, True), -1)]
  action=PICK    -> [(1.0, (3, 0, True), -3)]
  action=DROP    -> [(1.0, (3, 0, False), 20)]
```

STDERR:

Spreadsheet runtime warmup failed during python startup

Traceback (most recent call last):

```
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 772, in warm_spreadsheet
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/connection.py", line 37, in get_or_create_client
File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/daemon.py", line 124, in start_daemon
```

TimeoutError: Timed out waiting for artifact tool daemon socket. Set ARTIFACT\_TOOL\_RPC\_DAEMON\_STARTUP\_TIMEOUT\_S=<seconds>